



Research Description of the Accelera Parallelizer and Pipeline Modules

Mohamed Ashraf
ID: 9220658

Nesma Osama
ID: 9220912

Mazen Adel
ID: 9220625

Mohamed Khater
ID: 9220713

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework designed to combine Python's modelling productivity with C++ execution infrastructure. The project contains graph-based machine learning pipelines, parallel branch execution, HTML reporting, AutoML preprocessing utilities, dataset retrieval, deployment support, benchmarking infrastructure, and a Linux-only C/C++ loop parallelizer. This report is organized around five project modules: `accelera_pipe`, the code parallelizer, AutoML, deployment, and the benchmark platform. The `accelera_pipe` module constructs and executes directed machine learning workflows using a Python API over a C++ graph backend. The code parallelizer analyzes source loops, extracts features, classifies loops, and injects OpenMP pragmas. The remaining module sections are reserved for future expansion.

Contents

1	Project Introduction	3
2	Accelera Pipe Module	3
2.1	Module Location and Responsibility	3
2.2	Graph Backend	3
2.3	Pipeline Builder API	4
2.4	Execution Semantics	4
2.5	Preprocessing and Model Fitting	4
2.6	Prediction Nodes	4
2.7	Metric Nodes	4
2.8	Branching and Merging	5
2.9	Executed Graphs	5
2.10	Reporting	5
2.11	How to Use the Accelera Pipe Module	6
3	Code Parallelizer Module	7
3.1	Module Location and Responsibility	7
3.2	Native Loop Extraction	7
3.3	Feature Extraction	7
3.4	Vectorization for the Classifier	8
3.5	Loop Class Resolution	8
3.6	OpenMP Pragma Generation	8

3.7	Generator Model Trial and Rule-Based Design Rationale	8
3.8	Python-to-C++ Converter	9
3.9	Limitations	10
3.10	How to Use the Parallelizer	10
4	AutoML Module	12
5	Deployment Module	12
6	Benchmark Platform Module	12
7	Module-Level View	12
8	Conclusion	13

1 Project Introduction

Accelera targets experimental and production-oriented machine learning workflows where a user wants to express a pipeline in Python while delegating graph execution and branch scheduling to a compiled backend. Its main design idea is to represent preprocessing, model training, prediction, merging, and metric computation as graph nodes. Independent branches can then be evaluated in parallel, making it convenient to compare alternative preprocessors, estimators, and metrics in a single run.

The repository also includes a code parallelizer that operates outside the ML pipeline. That subsystem parses C/C++ source code through Clang AST bindings, extracts loop fragments, computes structural and dependency-related features, consults a remote classifier service, and inserts OpenMP `parallel` for pragmas where the loop appears safe or useful to parallelize. If the input file is Python, a small AST-based converter first emits equivalent C++ for a limited subset of Python syntax before loop analysis begins.

2 Accelera Pipe Module

2.1 Module Location and Responsibility

The `accelera_pipe` module is implemented under `accelera/src/accelera_pipe/`. It provides the Python API for building graph pipelines and report objects. Its core files are:

- `core/pipeline_base.py`: wraps the C++ `graph.Graph` object and exposes shared configuration methods.
- `core/pipeline.py`: public builder API for preprocessing, model, prediction, metric, merge, and branch nodes.
- `core/executed_graph.py`: reusable executed graph wrapper.
- `core/metric.py`: abstract metric interface.
- `wrappers/supervised_metric.py` and `wrappers/unsupervised_metric.py`: concrete metric adapters.
- `core/report_base.py` and wrapper display classes: HTML report generation for pipeline graphs and metrics.

2.2 Graph Backend

`PipelineBase` imports the native `graph` module. If this module is missing, it raises an import error that instructs the user to build the C++ bindings and make them available in `PYTHONPATH`. When initialized, the base class creates or reuses a `graph.Graph` instance and enables parallel execution by default:

```
self.__graph = _graph if _graph is not None else graph.Graph()
self.__graph.enableParallelExecution(True)
```

The node type mapping bridges Python method names to C++ enum values: `PREPROCESS`, `MODEL`, `PREDICT`, `METRIC`, and `MERGE`.

2.3 Pipeline Builder API

`Pipeline` is the main user-facing class. Each method either appends a node to the graph or, when `branch=True`, returns a lightweight `Node` object that can be supplied to `branch()`.

Table 1: Main `Pipeline` methods.

Method	Purpose	Important parameters
<code>preprocess</code>	Add a transformation step.	<code>name</code> , <code>func</code> , <code>branch</code> .
<code>model</code>	Add a trainable model.	<code>name</code> , <code>model</code> , <code>branch</code> .
<code>predict</code>	Add prediction over test data.	<code>test_data</code> , <code>output_func</code> , <code>positive_class</code> .
<code>metric</code>	Add a supervised or unsupervised metric.	<code>metric_name</code> , <code>y_true</code> , <code>plot_func</code> , <code>labels</code> , <code>headers</code> , extra metric parameters.
<code>merge</code>	Combine branch predictions.	<code>strategy</code> , currently defaulting to <code>hard_voting</code> .
<code>branch</code>	Split the graph into alternative paths.	A branch name and one or more <code>Node</code> objects created with <code>branch=True</code> .

2.4 Execution Semantics

Calling a `Pipeline` executes the underlying graph:

```
predictions, executed_graph = pipe(X, y, select_strategy="all")
```

The C++ backend returns an executed graph plus result values. The Python layer wraps the first returned item in `ExecutedGraph`; all remaining returned items are predictions or metric outputs. The selection strategy can be `all`, `max`, `min`, or a user-provided `custom_strategy`. Tests in the module show that these strategies are used to select paths according to metric outputs.

2.5 Preprocessing and Model Fitting

`Preprocess` nodes store the transformation object or function together with `execute_fit`, a helper that determines whether a component should be fit with `X` only or with both `X` and `y`. This lets the pipeline handle unsupervised transformers such as scalers and supervised transformers such as feature selectors. `Model` nodes similarly store the model object and fitting helper, allowing `sklearn`-compatible estimators and custom `Accelerate` estimators to participate in the graph.

2.6 Prediction Nodes

`predict(name, test_data, output_func="predict", positive_class=-1)` records the data on which the trained model should be evaluated. The `output_func` parameter can request outputs such as `predict` or `predict_proba`. When probabilities are used, `positive_class` can select one probability column.

2.7 Metric Nodes

The pipeline maps metric names to functions from `sklearn.metrics`. Then the `metric-class` helper inspects the metric function signature:

- A metric with true labels and predicted labels/scores becomes a **SupervisedMetric**.
- A metric with **X** and **labels** becomes an **UnSupervisedMetric**.
- Any incompatible signature is rejected.

SupervisedMetric.execute validates that **y_pred** and **y_true** have the same number of samples, then calls the sklearn metric. **UnSupervisedMetric.execute** validates predictions against the stored input **X**. Both return a dictionary containing the metric name, result, plot function, label names, and header names.

2.8 Branching and Merging

branch() accepts one or more branch node objects. Each branch can be a single node or an iterable of nodes. The method validates that all branch members are **Node** instances and rejects merge nodes inside branches because merge-in-branch behavior is not implemented yet. The validated branch objects, node types, and node names are passed to the C++ graph through **split()**.

Merge nodes combine multiple branch outputs. The default strategy is **hard_voting**. The test suite covers hard-voting merges, merges after preprocessing branches, probability prediction merges, single-model merges, and reuse of executed graphs after merging.

2.9 Executed Graphs

ExecutedGraph wraps a graph that has already been fitted or selected. Calling it with new **X** reuses the fitted pipeline. If **y_true** is provided, metrics are temporarily enabled for that call and disabled afterward. **save(directory)** serializes the pipeline XML to **pipeline.xml**; the code notes that model artifact saving is still a future improvement.

2.10 Reporting

The reporting layer creates HTML reports rather than PDF reports. The base class **ReportBase** defines the HTML shell and utility functions. **GraphReport** reads serialized pipeline XML, builds a Graphviz diagram, colors selected path nodes red and other nodes blue, lists branches, and then delegates metric rendering to **GraphPipelineReport**.

The graph-pipeline report groups metric records by metric name and chooses a display wrapper according to the result type:

- scalar values: **DisplaySingleNumber**;
- one-dimensional arrays: **DisplayArraySingle**;
- multi-dimensional arrays: **DisplayMultiArray** or **DisplayFigure**;
- dictionaries: **DisplayDict**;
- strings: **DisplayString**;
- tuples: **DisplayTupleNotCurve** or **DisplayFigure**.

ModelReport reuses this metric rendering and can optionally plot training history when a dictionary of history arrays is provided.

2.11 How to Use the Accelerate Pipe Module

A minimal classification pipeline is:

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from accelera.src.accelera_pipe.core.pipeline import Pipeline

X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_test, y_test = X[:100], y[:100]

pipe = Pipeline()
pipe.preprocess("scale", StandardScaler())
pipe.model("logreg", LogisticRegression(max_iter=1000))
pipe.predict("predict", test_data=X_test)
pipe.metric("accuracy", "accuracy_score", y_true=y_test)

results, executed_graph = pipe(X, y)
print(results)
```

A branching experiment compares preprocessors and models:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import MinMaxScaler, StandardScaler

pipe = Pipeline()
pipe.branch(
    "preprocessing",
    pipe.preprocess("standard", StandardScaler(), branch=True),
    pipe.preprocess("minmax", MinMaxScaler(), branch=True),
)
pipe.branch(
    "models",
    pipe.model("logreg", LogisticRegression(max_iter=1000), branch=True),
    pipe.model("forest", RandomForestClassifier(n_estimators=50), branch=True),
)
pipe.predict("predict", test_data=X_test)
pipe.metric("accuracy", "accuracy_score", y_true=y_test)

results, executed_graph = pipe(X, y, select_strategy="max")
best_result = executed_graph(X_test, y_test)
```

To generate a graph report:

```
from accelera.src.accelera_pipe.wrappers.graph_report import GraphReport
from accelera.src.utils.accelera_utils import serialize

results, executed_graph = pipe(X, y, select_strategy="max")
serialize(pipe, "pipeline.xml")

report = GraphReport("pipeline_report", "pipeline.xml", results)
report.execute()
```

3 Code Parallelizer Module

3.1 Module Location and Responsibility

The parallelizer is implemented mainly in `accelera/src/utils/parallelizer.py`. Its Python-to-C++ helper is in `accelera/src/utils/py2cpp_converter.py`. Native loop extraction is provided by the pybind module `code_parallelizer_utils`, which is built from C++ sources under `src/ast/` and `src/utils/code_parallelizer_utils.cpp`. The CMake configuration enables these bindings only on Linux with LLVM/Clang available.

Conceptually, the module implements the following pipeline:

1. Read a C, C++, or supported Python file.
2. Convert Python input to C++ when the file extension is `.py`.
3. Extract loops using Clang-backed native bindings.
4. Cache extracted loop metadata in `.accelera.cache`.
5. Compute a 40-dimensional feature vector for each `for` loop.
6. Send the vector to the classifier endpoint configured by `ACCELERA_CLASSIFIER_ENDPOINT`.
7. Resolve the final loop class with local heuristics.
8. Generate OpenMP pragmas and replace the original loop text.
9. Write `parallelized_<name>.c` beside the input file and format it with `clang-format` when available.

3.2 Native Loop Extraction

The public loop extraction helpers wrap the native `code_parallelizer_utils` bindings. The default Clang argument is `-std=c++17`. The expected extracted loop records include at least the loop source code, loop type, start line, and end line. Although the C++ AST layer can identify several loop kinds, the Python insertion logic currently parallelizes only loops whose extracted `type` is `for`.

3.3 Feature Extraction

The function `extract_features(code)` performs lightweight static analysis using regular expressions after comment removal. The extracted features are intentionally pragmatic: they describe loop structure, reduction patterns, memory access shape, control flow, and rough computational intensity. Important feature groups include:

- **Reduction features:** detection of scalar `+=`, `x = x + ...`, `*=`, `x = x * ...`, and textual `min/max` usage.
- **Dependency features:** loop-carried dependency patterns such as `a[i - 1]`, read-after-write flags, write-after-read flags, and dependency distance.
- **Memory features:** array reads, array writes, indirect indexing, number of memory operations, and hashed array-name embeddings.
- **Control-flow features:** branch count, early exits via `break`, `continue`, or `return`, and function call counts.

- **Shape features:** constant loop bounds, estimated iteration count, consecutive nested loop depth, and nested brace depth.
- **Vectorization hint:** a boolean flag that becomes true when the loop has no branches, no function calls, no loop-carried dependency, and no indirect access.

3.4 Vectorization for the Classifier

`vectorize_features(features)` maps the feature dictionary to a 40-element `numpy.float32` vector. Boolean features occupy positions 0–16. Counts and normalized scalar descriptors occupy later positions. Array names are transformed into compact deterministic hash vectors rather than sent as raw strings. This representation is sent as JSON to the remote classifier:

```
response = requests.post(
    classifier_endpoint,
    json={"embedding": features.tolist()},
    timeout=config.REQUEST_TIMEOUT_S,
)
```

The classifier result is expected to contain `result`, which is then interpreted as a predicted loop class.

3.5 Loop Class Resolution

The function `_resolve_loop_class(loop_code, pred_class)` combines remote prediction with local safety heuristics. If the classifier predicts a class other than `none`, that class is used. If it predicts `none`, the local logic can still upgrade the loop to:

- `reduction`, when scalar reduction updates are detected.
- `parallel_for`, when consecutive nested `for` loops are detected.

Otherwise the loop remains `none`. This means the module is not purely ML-based; it is a hybrid predictor with deterministic fallbacks for common OpenMP-friendly patterns.

3.6 OpenMP Pragma Generation

`_generate_omp_pragma_with_loop` creates the final annotation. For ordinary parallel loops it emits:

```
#pragma omp parallel for
for (...) {
    ...
}
```

For consecutive nested loops it adds `collapse(depth)`. For scalar addition or multiplication reductions it adds `reduction(+ : var)` or `reduction(* : var)`. The generator therefore currently focuses on OpenMP `parallel for`, `collapse`, and scalar reduction clauses.

3.7 Generator Model Trial and Rule-Based Design Rationale

An earlier generator trial attempted to learn OpenMP pragma generation as a sequence-to-sequence problem. In that formulation, the input sequence contained the loop source code prefixed by a class token such as `[CLS:parallel_for]` or `[CLS:reduction]`, and the target sequence was the expected OpenMP pragma. The implemented model was a custom PyTorch encoder-decoder network:

- **Encoder:** a bidirectional LSTM with embeddings, dropout, packed variable-length input handling, and layer normalization.
- **Decoder:** an LSTM decoder conditioned on Bahdanau additive attention over encoder states.
- **Bridge:** linear projections that map the bidirectional encoder hidden and cell states into the decoder state space.
- **Training:** teacher forcing, cross-entropy loss with padding ignored, Adam optimization, gradient clipping, and a ReduceLROnPlateau scheduler.

The deployed generator service in `models/open_mp_generator/app.py` uses the following configuration: embedding size 128, hidden size 256, three LSTM layers, dropout 0.2, greedy decoding, maximum input length 1500, and a checkpoint saved as `best_model.pth`. The tokenizer is a custom BPE tokenizer rather than a pretrained Hugging Face tokenizer. It is stored in `tokenizer.json`, uses the special tokens `<PAD>`, `<UNK>`, `<SOS>`, and `<EOS>`, and was configured for a target vocabulary of 8000 tokens. The saved tokenizer contains 8002 tokens and 7888 learned BPE merges.

Although this neural generator is expressive, two empirical limitations made a pure learned generator unsuitable for the current system. First, the available training data was small for a code-generation task and contained only a limited effective number of examples for some pragma classes. In particular, the corpus included about 15k negative examples and a comparatively narrow distribution of positive pragma patterns, which made the sequence model prone to memorization and overfitting. As a result, validation performance did not reliably imply generalization to unseen loop structures, variable names, dependency patterns, or clause combinations.

Second, expanding the corpus with AI-generated examples introduced distribution bias. Synthetic loops and pragmas tend to reflect the style, templates, and regularities of the generating tool. A model trained heavily on such data can learn those artifacts instead of learning robust OpenMP semantics. This lowers real-world generalization even when apparent training or validation error looks acceptable on similarly generated samples.

For these reasons, the implemented generator path was changed to a rule-based approach. OpenMP pragmas are now constructed from explicit static features such as consecutive loop depth and scalar reduction variables. This design is less flexible than open-ended neural generation, but it is more predictable, auditable, and aligned with the current objective: capture the most common and safe pragma forms, including `parallel for`, `collapse(n)`, and scalar `reduction` clauses, without hallucinating unsupported clauses or unbound variables.

3.8 Python-to-C++ Converter

The converter in `py2cpp_converter.py` uses Python’s built-in `ast` module. It does not attempt to be a complete Python compiler. Instead, it emits simple C++ for a restricted subset that is useful for loop experiments. Unsupported syntax raises `ValueError`. The output always contains an `int main()` for top-level statements, while Python function definitions are emitted before `main()` with `auto` return type and `auto` parameters.

Table 2: Supported operations and methods in the Python-to-C++ converter.

Category	Supported form	Generated C++ / Notes
Constants	<code>None</code> , <code>bool</code> , <code>int</code> , <code>float</code> , <code>string</code>	<code>nullptr</code> , <code>true/false</code> , numeric literals, escaped <code>std::string</code> literals.
Names	<code>x</code>	Emitted as the same identifier.

Category	Supported form	Generated C++ / Notes
Binary operations	<code>+, -, *, /, %</code>	Parenthesized infix expressions. <code>**</code> is rejected.
Unary operations	unary <code>+</code> , unary <code>-</code> , <code>not</code>	Mapped to <code>+</code> , <code>-</code> , and <code>!</code> .
Comparisons	<code>==, !=, <, <=, >, >=</code>	Only one comparison operator is allowed; chained comparisons are rejected.
Boolean operations	<code>and</code> , <code>or</code>	Mapped to <code>&&</code> and <code> </code> .
Function calls	<code>f(a, b)</code>	Emitted as ordinary C++ calls. Keyword arguments are generally unsupported.
Printing	<code>print(a, b)</code>	Mapped to <code>std::cout << a << " << b << std::endl</code> . Print keywords are rejected.
Attributes	<code>obj.attr</code>	Mapped to <code>obj.attr</code> .
Subscript	<code>arr[i]</code>	Mapped to <code>arr[i]</code> . Slices are rejected.
Assignments	<code>x = expr</code>	First assignment declares inferred type; later assignments reuse the variable. Multi-target and non-name assignments are rejected.
Augmented assignment	<code>x += y, x -= y, x *= y, x /= y, x %= y</code>	Simple-name targets only. <code>x += 1</code> is emitted as <code>x++</code> .
Return	<code>return</code> or <code>return expr</code>	Emitted directly.
Conditionals	<code>if/else</code>	Standard C++ braces.
For loops	<code>for i in range(...)</code>	Supports <code>range(stop)</code> , <code>range(start, stop)</code> , and <code>range(start, stop, step)</code> with simple name targets.
Functions	<code>def f(a, b): ...</code>	Emitted as <code>auto f(auto a, auto b) {...}</code> . Decorators are rejected.
Type inference	<code>bool, int, float, string</code> constants	Emits <code>bool</code> , <code>int</code> , <code>double</code> , or <code>std::string</code> ; otherwise <code>auto</code> .

3.9 Limitations

The parallelizer is Linux-oriented because it depends on LLVM/Clang development libraries and pybind bindings. It also depends on a remote classifier endpoint unless local integration is added. The Python-to-C++ converter supports only simple syntax, so real Python programs with classes, imports, comprehensions, lists, dictionaries, slices, exception handling, or NumPy calls will not convert directly. The feature extractor is regex-based, so it is fast and transparent but less semantically complete than full dependence analysis.

3.10 How to Use the Parallelizer

After installing LLVM/Clang and building the project bindings, a user can parallelize a C, C++, or supported Python file:

```
from accelera.src.utils.parallelizer import parallelizer

parallelizer.parallelize("examples/loop_example.py")
# Writes examples/parallelized_loop_example.c
```

A simple Python input that the converter can handle is:

```
total = 0
for i in range(0, 1000):
    total += i
print(total)
```

The converter emits C++; the parallelizer extracts the loop and can produce a result similar to:

```
#include <iostream>

int main() {
    int total = 0;
#pragma omp parallel for reduction(+ : total)
    for (int i = 0; i < 1000; i++) {
        total += i;
    }
    (std::cout << total << std::endl);
    return 0;
}
```

4 AutoML Module

5 Deployment Module

6 Benchmark Platform Module

7 Module-Level View

The five report modules describe different layers of Accelera. The `accelera_pipe` module focuses on graph-based machine learning workflow construction and execution. The code parallelizer focuses on source loop analysis and OpenMP generation. The AutoML, deployment, and benchmark platform sections are reserved for later module descriptions.

Across the implemented sections, a common architectural pattern appears: Python exposes a convenient API and performs validation, while compiled or external services provide lower-level parsing, execution, classification, or infrastructure support.

8 Conclusion

The five-module structure illustrates the central engineering style of Accelera: expose simple Python interfaces while using compiled infrastructure, model services, and supporting platforms for performance-sensitive or operational work. In the current report draft, `accelera_pipe` is documented as a graph-oriented ML workflow system with branching, metric adaptation, path selection, executed graph reuse, and HTML reporting. The code parallelizer is documented as a static-analysis and OpenMP generation tool with a restricted Python converter, feature extraction, classifier-based loop selection, and local heuristic fallbacks. The remaining sections are intentionally left as space for future documentation.